

# Exercise 4: Employee Management System

## 1. Aim

To develop an Employee Management System using Java arrays and implement operations such as adding, searching, traversing, and deleting employee records.

## 2. Problem Understanding

An Employee Management System stores and manages employee information such as Employee ID, Name, Position, and Salary.

Arrays are one of the simplest data structures used to store multiple elements of the same type. Since array elements are stored in **contiguous memory locations**, they provide fast access using an index.

Arrays are suitable when the number of records is fixed or changes infrequently.

## 3. Array Representation

An array stores elements in **contiguous memory locations**.

Example:

Index : 0    1    2    3    4

Array : [Emp1] [Emp2] [Emp3] [Emp4] [Emp5]

Each element is accessed directly using its index.

### Advantages of Arrays

- Fast access using index.
- Simple to implement.
- Efficient memory usage.
- Suitable for fixed-size collections.

## 4. Algorithm

### Add Employee

1. Check whether the array has space.
2. Insert the employee at the next available position.

### **Search Employee**

1. Traverse the array.
2. Compare the employee ID.
3. Display the employee if found.

### **Traverse Employees**

1. Visit every element in the array.
2. Display employee details.

### **Delete Employee**

1. Find the employee.
2. Shift all subsequent elements one position to the left.
3. Decrease the employee count.

## **5. Program/Output:**

```

J Exercise4.java > Exercise4 > displayEmployee()
1  class Employee{
2      int employeeId;
3      String name;
4      String position;
5      double salary;
6
7      Employee(int employeeId,String name,String position, double salary){
8          this.employeeId = employeeId;
9          this.name = name;
10         this.position = position;
11         this.salary = salary;
12     }
13
14     @Override
15     public String toString(){
16         return "EmployeeID:"+employeeId+", Name:"+name+
17             ", Position:"+position+
18             ", Salary: Rs."+salary;
19     }
20 }
21 public class Exercise4 {
22     static Employee[] employees = new Employee[10];
23     static int count =0;
24     public static void addEmployee(Employee employee){
25         if(count<employees.length){
26             employees[count++] = employee;
27             System.out.println(x: "Employee Added Successfully.");
28         }
29         else{
30             System.out.println(x: "Employee Array is full.");
31         }
32     }
33     public static void SearchEmployee(int id){
34         for(int i=0;i<count;i++){
35             {
36                 if(employees[i].employeeId==id){
37                     System.out.println(employees[i]);
38                     return;
39                 }
40             }
41             System.out.println(x: "Employee Not Found.");
42         }
43     public static void deleteEmployee(int id){
44         int index = -1;
45         for(int i=0;i<count; i++){
46             if(employees[i].employeeId==id){
47                 index = i;
48                 break;
49             }
50         }
51         if(index== -1){
52             System.out.println(x: "Employee Not Found.");
53             return;
54         }
55         for(int i=index;i<count-1;i++){
56             employees[i] = employees[i+1];
57         }
58         employees[count-1] = null;
59         count--;
60
61         System.out.println(x: "Employees Deleted Successfully");
62     }
}

```

```
69     public static void main(String args[]){
70         addEmployee(new Employee(employeeId: 101, name: "Krishna", position: "SE", salary: 60000));
71         addEmployee(new Employee(employeeId: 102, name: "vishwa", position: "GM", salary: 120000));
72         addEmployee(new Employee(employeeId: 103, name: "dharshith", position: "Manager", salary: 100000));
73
74         displayEmployee();
75
76         System.out.println(x: "\n Searching Employee:"
77     );
78
79     SearchEmployee(id: 102);
80
81     System.out.println();
82     deleteEmployee(id: 102);
83     displayEmployee();
84 }
85 }
86
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\Algorithm> & 'C:\Program Files\Java\jdk-17\bin\java.exe' '-X
lgorithm_cc0d2e2b\bin' 'Exercise4'
Employee Added Successfully.
Employee Added Successfully.
Employee Added Successfully.

Employee Records:
EmployeeID:101, Name:Krishna, Position:SE, Salary: Rs.60000.0
EmployeeID:102, Name:vishwa, Position:GM, Salary: Rs.120000.0
EmployeeID:103, Name:dharshith, Position:Manager, Salary: Rs.100000.0

Searching Employee:
EmployeeID:102, Name:vishwa, Position:GM, Salary: Rs.120000.0

Employees Deleted Successfully

Employee Records:
EmployeeID:101, Name:Krishna, Position:SE, Salary: Rs.60000.0
EmployeeID:103, Name:dharshith, Position:Manager, Salary: Rs.100000.0
```

## 6. Time Complexity Analysis

| Operation          | Time Complexity |
|--------------------|-----------------|
| Add Employee       | O(1)            |
| Search Employee    | O(n)            |
| Traverse Employees | O(n)            |
| Delete Employee    | O(n)            |

## 7. Limitations of Arrays

- Fixed size; cannot grow dynamically.
- Insertion and deletion require shifting elements.
- Searching is slow for large datasets.
- Memory is allocated in advance, which may waste space.

## When to Use Arrays

Arrays are suitable when:

- The number of elements is fixed.
- Fast index-based access is required.
- Memory usage needs to be simple and predictable.
- The application performs more reads than insertions or deletions.

## 8. Result

Thus, the **Employee Management System** was successfully implemented using Java arrays. Operations such as adding, searching, traversing, and deleting employee records were performed successfully. Arrays provide efficient index-based access but have limitations such as fixed size and costly insertion/deletion operations, making them suitable primarily for fixed-size datasets.